Nombre: Patricio Fernando Colunga Villarreal	**Matrícula:** 7193824
Nombre del curso: Desarrollo FullStack	**Nombre del profesor:** Luis Carlos Contreras Beltrán
Actividad / Proyecto Actividad 2	**Descripción:** **En esta actividad, desarrollarás una aplicación interactiva de gestión de tareas utilizando JavaScript, que permitirá a los usuarios agregar, editar y eliminar tareas directamente en la página *web*. Deberás estructurar el código utilizando clases y otras características modernas como *let*, *const*, funciones flecha, *template literals*, entre otras.** **La actividad se enfocará en la manipulación del DOM y el uso de características modernas de JavaScript como ES6+ y programación orientada a objetos (POO). La interfaz se construirá con HTML y CSS básicos; además la funcionalidad interactiva será implementada en JavaScript, haciendo uso de las técnicas más aprendidas en el curso.**
Fecha: 7 de noviembre de 2026	
Referencias bibliográficas:	

1. Introducción y Arquitectura

Para el desarrollo de esta actividad, se creó una aplicación web interactiva llamada **TaskMaster Pro**, la cual implementa un modelo **CRUD** (Create, Read, Update, Delete) completo. La interfaz fue construida con HTML y CSS, aplicando un diseño moderno, responsivo y accesible. Toda la lógica de interactividad se manejó a través de JavaScript, utilizando Programación Orientada a Objetos (POO) y manipulación del DOM. Se implementó la carga de imágenes adjuntas a cada tarea, aplicando técnicas de redimensionamiento y compresión en el lado del cliente.

2. Programación Orientada a Objetos (POO)

Siguiendo las mejores prácticas para mantener un código modular y escalable, la lógica se dividió en dos clases principales:

- **Clase** Tarea**:** Es el molde para cada tarea individual. Su constructor recibe el nombre, una imagen opcional (en formato Base64), el estado de completada y genera un ID único basado en Date.now(). Incluye métodos encapsulados para alterar su estado y editar su título.
- **Clase** GestorDeTareas**:** Es el cerebro de la aplicación. Administra el arreglo de tareas, se encarga de la manipulación del DOM (renderizar las tarjetas), maneja los eventos de los botones y gestiona la persistencia de los datos en el navegador.

3. Uso de Características ES6+

El código hace un uso intensivo de las herramientas modernas de JavaScript:

- **let** y **const:** Se utilizaron para declarar variables de manera segura. Las constantes se usan para elementos del DOM que no cambian.
- **Funciones Flecha (**=>**):** Empleadas en los *callbacks* de los eventos para mantener el contexto léxico de this.
- **Template Literals (`):** Utilizados en el método render() para inyectar dinámicamente el HTML de las tareas sin necesidad de concatenar strings.
- **Métodos de arreglos:** Uso de forEach para iterar sobre las tareas y filter para eliminarlas.

4. Manipulación del DOM y Gestión de Eventos

- **Prevención de errores:** En el evento submit se aplica **e.preventDefault()** para evitar que la página se recargue. En los botones de la lista, se utiliza **e.stopPropagation()** para evitar que al hacer clic en "Eliminar", se marque la tarea como completada por error.
- **Validación de datos:** Antes de agregar una tarea, se valida con .trim() que el usuario no haya ingresado solo espacios en blanco.
- **Solución de errores de consola:** Se implementó la carga del script con el atributo defer en el HTML, lo que garantiza que el DOM esté completamente cargado

antes de que JavaScript intente seleccionar los elementos (evitando el error TypeError: Cannot set properties of null).

5. Gestión Avanzada de Imágenes

Para permitir adjuntar imágenes sin saturar el LocalStorage, se implementó un sistema de compresión:

1. **Lectura:** Se usa FileReader para leer el archivo local.
2. **Redimensionamiento:** Al cargar la imagen, se utiliza un elemento dinámico <canvas> de HTML5. Se calculan nuevas proporciones (máximo 400px) para evitar deformaciones.
3. **Compresión Base64:** La imagen en el canvas se exporta con canvas.toDataURL('image/jpeg', 0.7). Esto reduce el peso de megabytes a unos pocos kilobytes y permite guardarla como texto.

6. Persistencia de Datos (Desafío Adicional)

Se implementó el uso de localStorage para que las tareas persistan al recargar.

- **Guardar:** Se utiliza JSON.stringify() y se envuelve en un bloque try...catch para manejar el error si el almacenamiento se llena.
- **Cargar:** Al iniciar, se utiliza JSON.parse() para recuperar la información y se vuelven a instanciar los objetos con la clase Tarea.

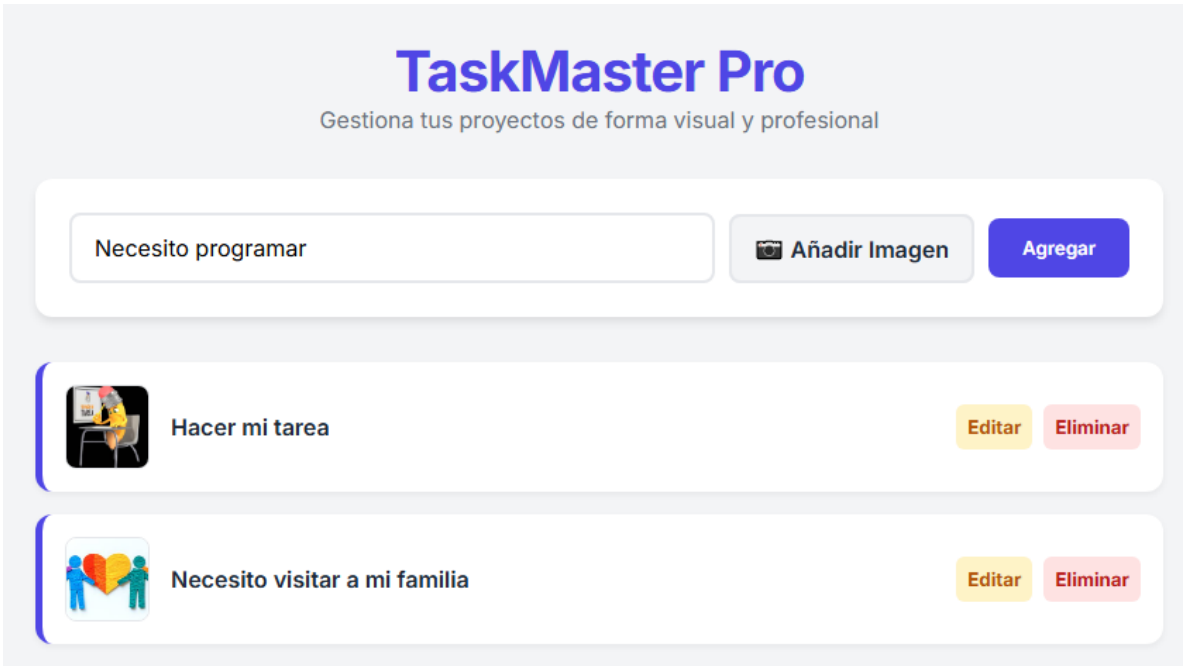
7. Diseño Responsivo y UX/UI

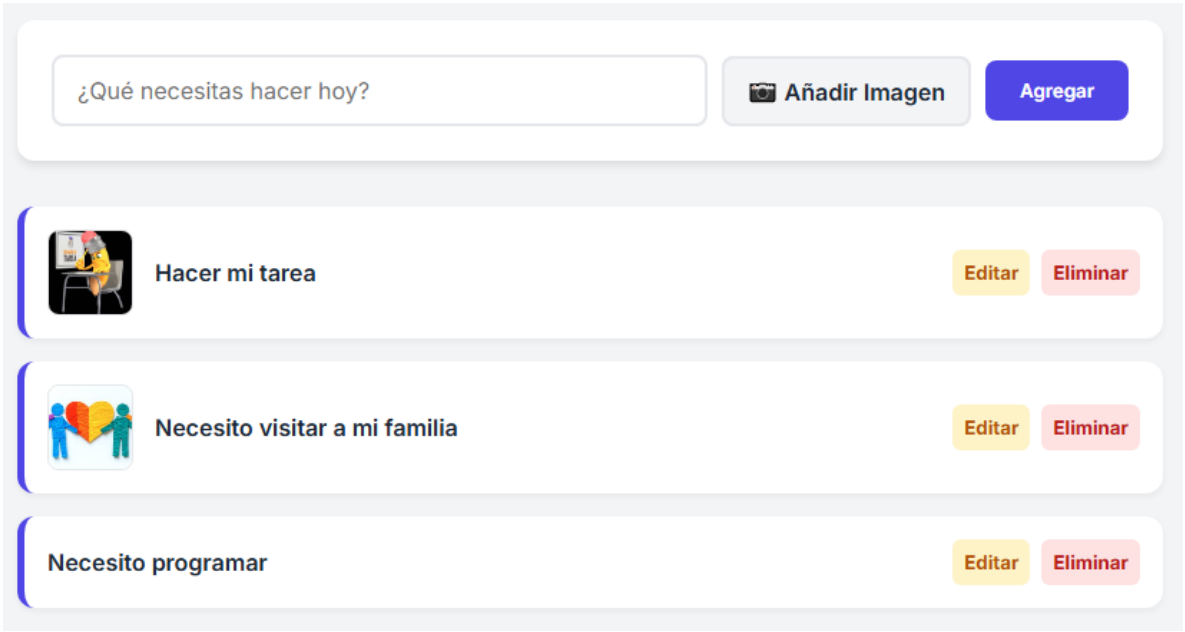
- Se utiliza <meta name="viewport"> para adaptarse a móviles.
- Se aplicaron **Media Queries** (@media (max-width: 600px)) para que el formulario y los botones se apilen verticalmente en pantallas pequeñas.
- Se utilizaron **Variables CSS** (:root) para una paleta de colores consistente, bordes redondeados, sombras suaves y transiciones (hover) que dan una sensación profesional.

Pruebas de la aplicación

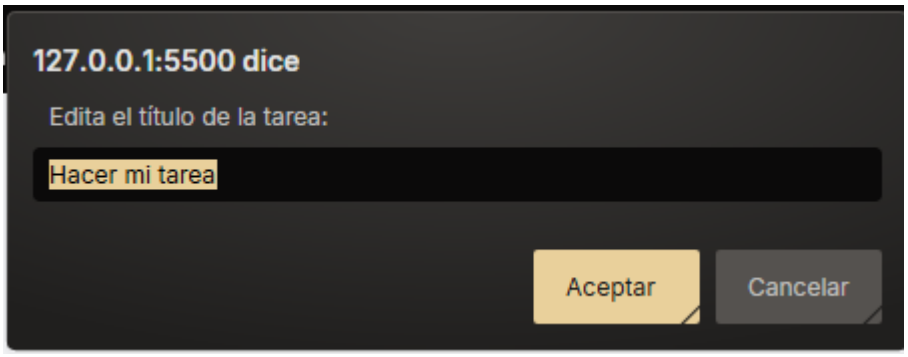


Create: Se escribe el nombre de la tarea y opcionalmente se añade una imagen. Despues se añade con el botón “Agregar”



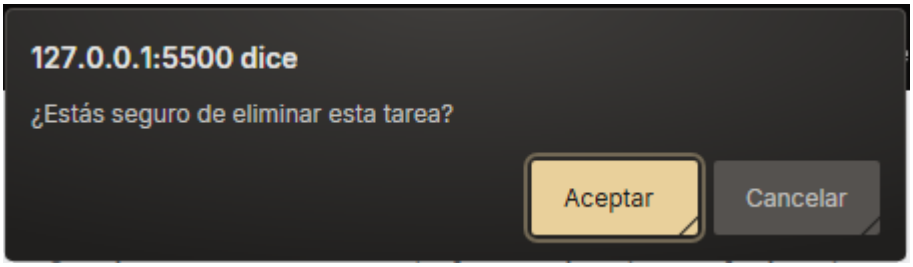


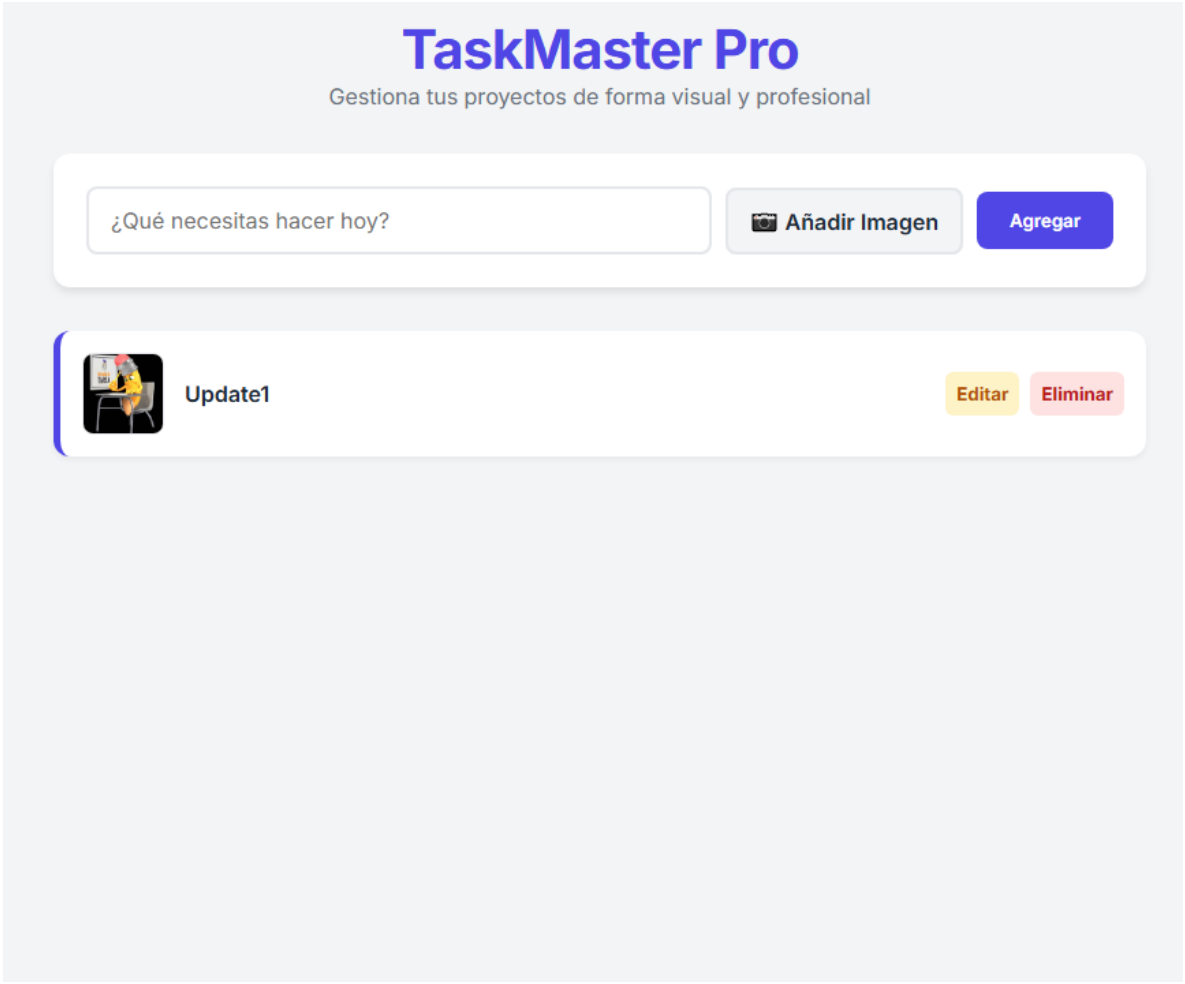
Update: al hacer clic en editar se abre una ventanita para modificar el nombre.



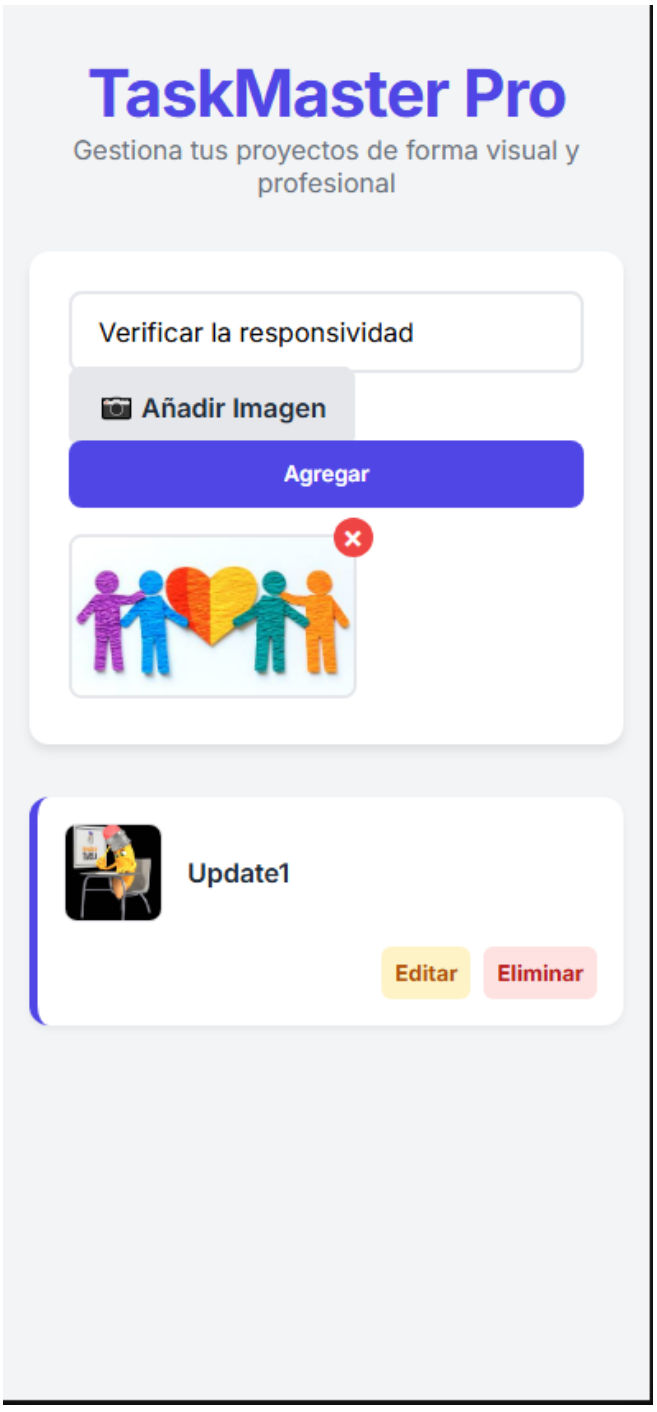


Delete: Al hacer click en el botón eliminar, puedes borrar la tarea seleccionada





Diseño responsivo (iphone 16)



TaskMaster Pro

Gestiona tus proyectos de forma visual y profesional

¿Qué necesitas hacer hoy?

📷 Añadir Imagen

Agregar



Update1

Editar Eliminar



Verificar la responsividad

Editar Eliminar